

Ex13 — v-on : Bouton Liker

Informations

Notion	Gestion des événements — v-on (@click)
Niveau	● Facile
Durée estimée	15 minutes
Travail	Individuel — sans aide IA

Objectif

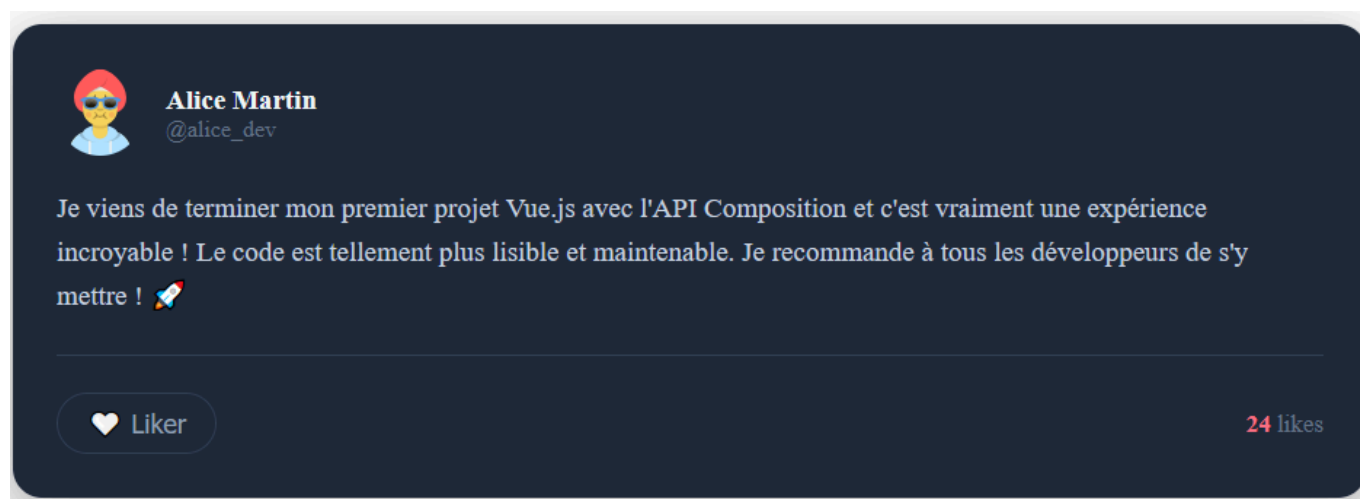
Rendre interactif un bouton "Liker" en utilisant @click pour écouter les événements et modifier des données réactives.

Le HTML et le CSS sont fournis — vous ne devez écrire que la **logique Vue.js**.

Structure de l'exercice

```
/ex13-bouton-liker
├── src/
│   └── App.vue
└── index.html
```

Résultat attendu



Ce qui est fourni

Copiez ce code dans `App.vue` — le HTML et le CSS sont déjà en place. Vous n'avez qu'à compléter le `<script setup>` et modifier le `<template>`.

```
<script setup>
// À compléter
</script>

<template>
  <div class="card">
    <div class="author">
      <img src="" alt="" class="avatar" />
      <div>
        <div class="author-name">authorName</div>
        <div class="author-handle">authorHandle</div>
      </div>
    </div>

    <p class="content">content</p>

    <div class="actions">
      <button class="btn-like">
         Liker
      </button>
      <div class="likes-count">
        <span>likes</span> likes
      </div>
    </div>
  </div>
</template>

<style scoped>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Arial', sans-serif;
  background: #0f172a;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 2rem;
}

.card {
  background: #1e293b;
  border-radius: 16px;
  padding: 1.5rem;
  min-width: 400px;
}
```

```
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
  }

  .author {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    margin-bottom: 1rem;
  }

  .avatar {
    width: 48px;
    height: 48px;
    border-radius: 50%;
    object-fit: cover;
  }

  .author-name {
    color: #f1f5f9;
    font-weight: bold;
    font-size: 0.95rem;
  }

  .author-handle {
    color: #64748b;
    font-size: 0.8rem;
  }

  .content {
    color: #cbd5e1;
    font-size: 0.95rem;
    line-height: 1.6;
    margin-bottom: 1.25rem;
    padding-bottom: 1.25rem;
    border-bottom: 1px solid #334155;
  }

  .actions {
    display: flex;
    align-items: center;
    gap: 1rem;
  }

  .btn-like {
    background: #1e293b;
    border: 1px solid #334155;
    border-radius: 999px;
    padding: 0.4rem 1rem;
    color: #94a3b8;
    font-size: 0.9rem;
    cursor: pointer;
    transition: all 0.2s;
  }
```

```
.btn-like.liked {
  background: #4c0519;
  border-color: #e11d48;
  color: #fb7185;
}

.likes-count {
  color: #64748b;
  font-size: 0.85rem;
  margin-left: auto;
}

.likes-count span {
  color: #fb7185;
  font-weight: bold;
}
</style>
```

À faire

- ☒ Définir les données réactives avec `ref()` :
 - `authorName`, `authorHandle`, `authorImage`, `content`
 - `likes` — ex : 24
 - `isLiked` — booléen, `false` par défaut
- ☒ Remplacer les textes en dur par `{{ }}`
- ☒ Lier `:src` et `:alt` sur l'avatar
- ☒ Définir une fonction `toggleLike()` qui incrémente/décrémente `likes` et bascule `isLiked`
- ☒ Lier `@click="toggleLike"` sur le bouton
- ☒ Appliquer la classe `liked` avec `:class="{ liked: isLiked }"`
- ☒ Changer le texte du bouton selon `isLiked` : ❤️ Liké ou 💜 Liker
- ☒ Pas d'utilisation d'IA pour générer le code

💡 **Test** : Cliquez sur le bouton et vérifiez que le compteur alterne et que le style change correctement.